

Efficient BERT Inference on x86 CPUs with AITemplate

摘要 | 本專題在原先以 CUDA/ROCm 為主要執行目標的 AITemplate 中加入 x86 CPU backend，並以 BERT-family inference 驗證其可行性。實作包含 CPU code generation、XNNPACK 整合，以及 static weight prepacking、temporary buffer reduction 與 operator fusion。實驗結果顯示，BERT-base 與 BERT-large 的推論延遲皆低於 PyTorch CPU baseline，其中 BERT-large 的 peak RSS 約降低 19.5%

1. 研究動機與問題

AITemplate 是 Meta 開源的 deep learning compiler，原始公開架構主要以 NVIDIA CUDA 與 AMD ROCm GPU 為執行目標。其 frontend 負責 graph transformation，backend 則依照目標硬體產生對應的 C++ 與 kernel code。[1] 由於原始 operator implementation、backend 與部分 runtime interface 都建立在 GPU execution 的假設上，因此若要支援 x86 CPU，除了新增 CPU kernel，也必須處理 code generation、runtime 介面與記憶體使用方式。

深度學習編譯器通常還需要處理不同硬體上的效能差異。比如 TVM 透過 graph-level 與 operator-level optimization 支援多種硬體 target；oneDNN Graph Compiler 則將 optimized kernels 與 graph compilation 結合，並針對 constant weights、operator fusion 與 memory-buffer reuse 進行最佳化。[2][3] 這些工作顯示，CPU inference 的效能不只取決於單一 kernel，也會受到整體 graph 與資料搬移成本影響。

本專題因此以保留 AITemplate 原有 graph compilation 與 code-generation 架構為前提，加入 x86 CPU backend，並聚焦以下三個問題：

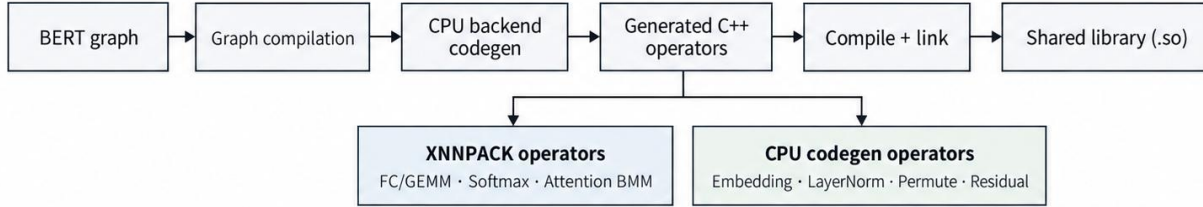
- 如何將 BERT 所需 operators 映射到 x86 CPU，並沿用 AITemplate 既有的編譯與執行流程。
- 在使用 optimized CPU kernels 後，static weight preparation、temporary memory 與額外 tensor pass 是否仍會影響 end-to-end latency。
- 能否利用編譯時已知的 tensor shape、constant weight 與 graph dependency，進一步減少上述 overhead。

BERT 被選為主要 workload，除了原始專案已提供測試程式碼範本，也因為其 encoder 同時包含大量 Fully Connected / GEMM、Attention、Softmax、ApproxGELU、Residual 與 LayerNorm，可同時觀察運算量較大的 operator 與記憶體存取成本。[5]

2. 系統設計與實作

本專題大致保留 AITemplate 原有 frontend 與 graph compilation 流程，新增 x86 CPU target 與 backend code generator，使模型產生 CPU C++ code，並編譯、連結成 shared library (.so)。BERT 路徑中，FP32 Fully Connected / GEMM、Softmax，以及 Attention 的 QK^T / AV BMM 使用 XNNPACK；LayerNorm、Embedding、permutation 與 Residual 等則由 CPU backend 產生 C++ 實作。[4]

Compile time



Runtime

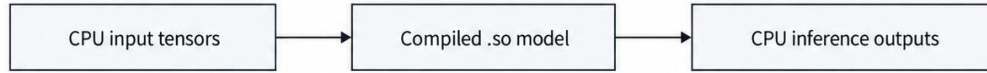


圖 1 AI Template x86 CPU backend 的編譯與執行流程

AI Template frontend 先完成 graph compilation，CPU backend 再產生 operators：部分呼叫 XNNPACK，部分直接產生 CPU C++。最後編譯、連結成 shared library (.so)，推論時由 CPU runtime 呼叫。

完成 BERT 所需的 CPU operators 後，模型已能完整執行；進一步 profiling 則顯示，整體推論時間除了主要計算本身，也受到權重準備、暫存資料與額外 tensor 讀寫等執行成本影響。這些操作會隨 BERT 各層反覆出現，因此後續最佳化除了維持底層 kernel 的效率，也進一步檢視哪些準備工作可以重用、哪些資料讀寫可以在既有流程中合併完成。

表 1 主要效能問題與對應實作方式

問題	實作方式
Static FC weight 固定，但 inference 時仍會重做相同的 weight preparation	Static weight prepacking：只建立一次 XNNPACK FC operator/context，並以 deterministic cache ID 在後續 inference 重用
FFN / Residual + LayerNorm 會建立只供下一步使用的 temporary buffers	In-place ApproxGELU；GEMM、Residual、LayerNorm 共用 output buffer，減少中間 buffer 與額外 tensor 讀寫
QKV copy 後，Q scaling 需要再完整讀寫一次 Q tensor	將 Q scaling 併入 QKV permutation / copy，在 copy Q 時同步完成 scaling
部分 CPU operators 需要高效率的底層 kernel	FC/GEMM、Softmax、Attention BMM 使用 XNNPACK；其餘由 CPU backend 產生 C++ 實作

2.1 Static weight prepacking

BERT 的主要 Fully Connected (FC) weights 在模型載入完成後便保持固定。但在最初的 CPU backend 實作中，XNNPACK FC operator/context 的建立位於 operator 的執行路徑內，因此每次 inference 執行到該 FC 時，都會重新建立與同一組 weight 有關的執行狀態。這裡的 weight-dependent state，是指 XNNPACK 會依據 weight 的內容與 shape 建立 FC operator/context，並將原始 weight 整理成較適合底層 kernel 使用的 packed format。若每次 inference 都重新建立，就會對固定不變的 weight 重複進行相同的準備工作。

本專題因此加入 static weight prepacking：在模型執行狀態初始化時完成 XNNPACK FC operator/context 與 packed weight 的建立，之後的 inference 直接重用。為了讓同一組 weight 能穩定對應到同一份 prepacked context，以 tensor name 產生 deterministic cache ID，而不再依賴執行時可能改變的 raw pointer 位址。完成

prepack 後，部分不再需要的 raw constant storage 也可釋放，只保留後續推論需要的 packed state，以降低記憶體占用。

2.2 Memory reuse 與 operator fusion

除了 FC 的準備成本外，profiling 也顯示 BERT 執行路徑中存在多塊只用於銜接相鄰 operators 的 temporary buffer。這類 buffer 不是模型最終需要保留的結果，而是上一個 operator 先寫入中間資料，下一個 operator 再將整份 tensor 讀出處理。對 BERT 的大型 tensor 而言，即使其中的運算本身不複雜，額外的一次完整讀寫仍會增加 memory traffic，並提高推論期間的記憶體需求。

在 FFN 中，原本 GEMM 產生結果後，ApproxGELU 還需要再讀取並寫入另一塊 activation scratch。修改後讓 ApproxGELU 直接在 GEMM output buffer 上 in-place 執行，因此不再配置這塊 scratch。Residual + LayerNorm 路徑則讓 GEMM+bias 直接寫入後續使用的 output buffer，再於同一個 buffer 上完成 residual addition 與 LayerNorm，移除原先 GEMM-sized temporary buffer。

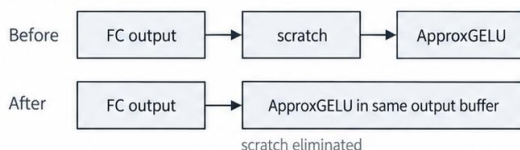
Attention 路徑也有類似的資料搬移成本。原本流程為 QKV GEMM → QKV permutation / copy → Q scaling → QK^T ；在 QKV copy 完成後，Q scaling 需要再完整走訪一次 Q tensor。由於 scaling 只是將 Q 的每個元素乘上固定係數，修改後便在 copy Q 的同時完成 scaling，使 QK^T 可以直接使用處理後的 Q，省去一次額外的 full-tensor pass。此融合僅在 compile time 能確認為 BERT attention path 且 scale 已知時套用，其他 BMM caller 仍保留原路徑。

A. Static FC weight prepacking

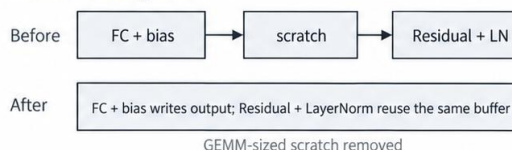


B. Reduce temporary buffers

FFN activation



Residual + LayerNorm



C. Fuse Q scaling into QKV copy

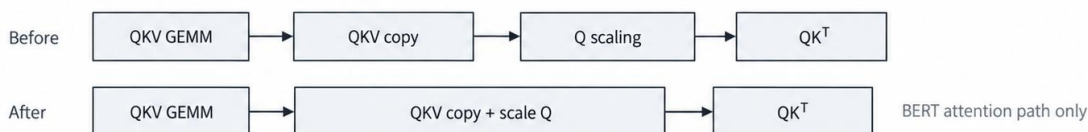


圖 2 Static weight prepacking、memory reuse 與 operator fusion 的實作概念

圖中三個區塊是獨立的 BERT 最佳化路徑：static FC 重用 prepacked context；FFN / residual path 以 in-place 與共用 output buffer 減少 temporary buffers；Attention 將 Q scaling 併入 QKV copy pass。

整體而言，這些最佳化建立在 AITemplate 編譯階段已能取得 constant weight、tensor shape 與 graph dependency 的基礎上。透過這些資訊，可以將固定不變的準備工作移出反覆執行的 inference 路徑，並把可

同時完成的 tensor 操作併入既有資料搬移流程，在不改變模型計算結果的前提下減少額外執行與記憶體成本。

3. 實驗設計與結果

實驗平台為 Intel Core i9-12900H，固定使用 P-cores；batch size 為 1、sequence length 為 128，資料型態為 FP32。比較對象為 PyTorch CPU inference，測試 BERT-base、BERT-large 與 Megatron-BERT 1.3B。Latency 以多次 isolated rounds 的 median-of-medians 統計，並使用 deterministic synthetic FP32 weights，使 AITemplate 與 PyTorch 使用相同 tensor。

表 2 AITemplate x86 backend 與 PyTorch CPU inference latency

Model	Threads	AITemplate	PyTorch	Speedup
BERT-base	1	207.29 ms	230.94 ms	1.114×
BERT-base	4	69.07 ms	75.24 ms	1.089×
BERT-large	1	725.25 ms	772.61 ms	1.065×
BERT-large	4	254.09 ms	271.90 ms	1.070×
Megatron-BERT 1.3B	1	2835.89 ms	2889.43 ms	1.019×
Megatron-BERT 1.3B	4	974.21 ms	992.63 ms	1.019×

BERT-base 的 1-thread 與 4-thread speedup 分別為 1.114× 與 1.089×；BERT-large 分別為 1.065× 與 1.070×。Megatron-BERT 1.3B 在 1-thread 與 4-thread 下皆為 1.019×，改善幅度已明顯縮小。

表 3 Median Peak RSS

Model	Threads	AITemplate	PyTorch	Memory Reduction
BERT-base	1	607.34 MiB	684.87 MiB	11.3%
BERT-base	4	610.46 MiB	691.11 MiB	11.7%
BERT-large	1	1556.72 MiB	1934.70 MiB	19.5%
BERT-large	4	1556.94 MiB	1935.22 MiB	19.5%
Megatron-BERT 1.3B	1	5160.11 MiB	5540.12 MiB	6.9%
Megatron-BERT 1.3B	4	5160.09 MiB	5539.65 MiB	6.9%

Peak RSS 在 BERT-base 上約降低 11.3%（1-thread）與 11.7%（4-thread），BERT-large 約降低 19.5%，Megatron-BERT 約降低 6.9%。此結果與 static weight storage 與 intermediate buffers 的最佳化方向一致；不過目前尚未完成逐項 ablation，因此無法將 memory reduction 完全歸因於單一修改。

4. 討論

模型規模增加後，static weight prepacking、scratch reduction 與 fusion 所帶來的 latency 改善逐漸縮小。BERT-base 與 BERT-large 仍有穩定改善；Megatron-BERT 1.3B 的 profiling 則顯示時間更集中於大型 GEMM，因此 graph-level memory optimization 的相對效益下降。

這代表目前 CPU backend 的主要瓶頸已逐漸從周邊 overhead 轉向 matrix multiplication kernel 本身。oneDNN Graph Compiler 也採用 optimized primitives 與 compiler-level optimization 並行的方式，以同時處理高成本 kernel 與 graph-level overhead。[3] 對本專題而言，後續若要在更大型模型上取得明顯加速，除了維持既有的 memory 與 fusion 最佳化外，還需要進一步處理 GEMM kernel、資料型態或 shape-specific tuning。整體結果顯示，AITemplate 原有的 graph 與 code-generation 架構可延伸至 x86 CPU，且編譯階段已知的 static weights、tensor shape 與 graph dependency 可實際用於減少 weight preparation、buffer allocation 與額外 tensor pass。

5. 研究限制與結論

目前 backend 仍屬實驗性實作，主要支援 FP32 與 BERT-family 所需 operators，尚未形成完整的 general-purpose CPU backend；部分 upstream frontend 與 runtime interface 也仍保留 GPU assumptions。此外，目前實驗集中在 Intel Core i9-12900H、batch size 1 與 sequence length 128，尚未涵蓋不同 CPU microarchitecture、batch size、sequence length 與其他 Transformer architectures。

本專題完成 AITemplate x86 CPU backend 的基本 code generation 與 BERT execution path，並實作 static weight prepacking、scratch buffer reuse / elimination、in-place ApproxGELU、Residual + LayerNorm overlay，以及 Q scaling 與 QKV permutation fusion。BERT-base 與 BERT-large 的 latency 均低於 PyTorch CPU baseline，BERT-large 亦觀察到較明顯的 peak RSS 降低；模型擴大至 Megatron-BERT 後，改善幅度縮小，profiling 顯示大型 GEMM 已成為主要瓶頸。後續工作可進一步針對 GEMM、低精度資料型態與更多 Transformer 模型進行實驗。

參考文獻

- [1] Meta AI, “Faster, more flexible inference on GPUs using AITemplate, a revolutionary new inference engine,” Oct. 3, 2022.
- [2] T. Chen et al., “TVM: An Automated End-to-End Optimizing Compiler for Deep Learning,” in 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2018, pp. 578–594.
- [3] J. Li et al., “oneDNN Graph Compiler: A Hybrid Approach for High-Performance Deep Learning Compilation,” in 2024 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), 2024, pp. 460–470.
- [4] Google, “XNNPACK: High-efficiency floating-point neural network inference operators for mobile, server, and Web,” GitHub repository.
- [5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in NAACL-HLT, 2019, pp. 4171–4186.